Continuous Patient Vital-Sign Monitoring with Fog-Layer Early Warning: A Fog-to-Cloud Architecture

Sri Venkat Bora
MSc in Cloud Computing
Fog and Edge Computing (H9FECC)
National College of Ireland
Student ID: X25164414
x25164414@student.ncirl.ie

***Abstract*—Clinical deterioration is usually preceded by measurable drift in a patient's vital signs, yet on a busy ward those signs may be recorded only intermittently, so a developing crisis can go unseen between observations. This paper presents a working fog-to-cloud pipeline that monitors patient vital signs continuously and applies early-warning logic at the edge. Each monitored patient carries five instruments, heart rate, blood-oxygen saturation, body temperature, respiration rate, and systolic blood pressure, that are sampled locally and dispatched to a fog gateway at the bedside. At the gateway each stream is placed in a fixed window, condensed to summary statistics, and screened against two-sided early-warning thresholds, so that a value that is abnormally high or abnormally low, a tachycardia or a bradycardia, a fever or hypothermia, is flagged in the cycle it occurs. Each window's compact aggregate then crosses a managed message queue to an event-driven ingestion function that stores it, while a separate, independently deployed function answers a dashboard interface behind a managed gateway and the operator page is served as static content from object storage, presenting each patient's heart rate as a live trace. The system is built in Node.js and provisioned to a real Amazon Web Services account by a single infrastructure-as-code apply that created twenty-four resources. Taken from the live endpoint, the measurements show the pipeline healthy end to end within about a minute of start-up and the stored record count rising across polls, with both patients rendering live in a browser and a hypoxia alert firing on one of them.**

***Index Terms*—fog computing, edge computing, patient monitoring, early warning score, serverless, message queue, Internet of Things.**

I. Introduction

Adverse events in hospital wards, cardiac arrests, unplanned intensive-care admissions, are rarely sudden. They are typically preceded by hours of measurable physiological drift, and structured early-warning scores were developed precisely to catch that drift: by assigning points to how far each vital sign strays from a normal band and summing them, a bedside team can be prompted to escalate before a patient collapses [1]. The National Early Warning Score, now widely standardised, formalises the vital signs to track and the bands that define normality [2]. The limitation these scores share is not the logic but its cadence: a score is only as current as the last set of observations, and on a busy ward those may be taken only every few hours, leaving long intervals in which deterioration is invisible.

Continuous instrumentation removes that blind interval, and connected medical sensing has matured to the point where a patient's vital signs can be streamed and analysed in real time [3]. But the architectural question remains: where should the analysis run? Streaming every raw sample to a central cloud is a poor default. A handful of vital-sign sensors per patient produce readings far faster than any single reading is clinically meaningful, the analysis that matters, is this window abnormal, is simple and local, and a monitoring view that stalls whenever connectivity dips is unacceptable at a bedside. Edge and fog computing respond by relocating computation toward the point where the data arises [4]. Interposing a fog tier between the instruments and the cloud keeps the windowing, the statistical reduction, and the early-warning checks at the bedside, and reserves for the cloud what it is suited to, durable storage, elastic query serving, and an interface reachable worldwide [5], [6]. The cloud services used here conform to the standard on-demand, elastic, metered definition of cloud computing [7].

This paper reports the design, implementation, and live deployment of such a pipeline. The deployment monitors two patients, each carrying five instruments. The objectives are fourfold. First, to reduce raw samples to compact per-window aggregates at a fog tier at the bedside. Second, to screen each window against two-sided early-warning thresholds at the edge, so that a deviation in either direction is flagged in the cycle it appears. Third, to build a scalable cloud back end that ingests aggregates reliably, stores them durably, and serves them to a live operator dashboard without the write path and the read path contending. Fourth, to deploy the whole system to a real cloud account and verify it end to end. The rest of the paper presents the architecture and the reasoning behind it, the implementation and its tests, the deployment, an evaluation from the running system, and a concluding reflection.

II. System Architecture and Design

The design is a single chain of ingestion with a branch that serves the clinician's view. Readings originate at the instru-

ments; the bedside fog gateway divides them into windows, condenses them, and checks them against thresholds; a queue conveys the resulting summaries to the cloud; one event-driven function commits them to storage; and a separate function reads them back for the dashboard. The deployment is laid out in Fig. 1. Each tier is examined below, and the section ends by weighing the alternatives.

A. Bedside Sensing and the Fog Gateway

Five independent sensor processes stand in for each patient, one for each vital sign: heart rate in beats per minute, blood-oxygen saturation as a percentage, body temperature in degrees Celsius, respiration rate in breaths per minute, and systolic blood pressure in millimetres of mercury. Each sensor produces a realistic signal through a bounded random walk, advancing every reading by a small random amount from its predecessor and confining it to a physiologically plausible range, so the trace drifts naturally instead of jumping. Two intervals, set independently, control it: one determines how often a reading is produced, the other how often the collected readings are forwarded to the gateway. Keeping the two apart allows a quickly changing vital to be read frequently and still reach the edge in economical batches. What each forward carries is a compact JSON body identifying the vital, the patient, and the unit, together with the short list of timestamped readings gathered since the last forward, a payload of a few hundred bytes.

The fog gateway is a lightweight Node.js service on Express. It receives the posted batches and maintains a single running accumulator for each vital-and-patient pair. It does not keep the raw readings; each value, on arrival, is folded into a compact tuple of count, running sum, minimum, maximum, and latest reading, so the gateway's memory does not grow with the number of readings. At a fixed interval it closes the current window, converting each accumulator into a summary bearing the window bounds, the count, the minimum, the maximum, the mean, and the latest reading, and screening that summary against the early-warning thresholds for its vital before passing it on.

B. Edge Early-Warning Rules

Unlike a fault that only manifests in one direction, a vital sign can be dangerous when it is too high or too low, so most vitals here are screened against a two-sided threshold. Table I lists the rules and the clinical condition each encodes. Nine conditions are evaluated across the five vitals: bradycardia and tachycardia on heart rate, hypoxia on oxygen saturation, fever and hypothermia on temperature, bradypnea and respiratory distress on respiration, and hypotension and hypertension on blood pressure. Most rules test the window mean, because a sustained average is a more reliable signal than one noisy sample; the hypothermia rule instead tests the window minimum, because the coldest reading in the window is the safety-relevant one. Screening at the edge means a deviation is flagged in the same cycle as the readings that reveal it, without a round trip to the cloud, so the early-warning logic keeps pace with the data.

TABLE I
EDGE-EVALUATED EARLY-WARNING RULES

Vital	Rule (per window)	Alert raised
Heart rate	mean below 50 / above 120 bpm	Bradycardia / tachycardia
Oxygen saturation	mean below 92 %	Hypoxia
Body temperature	mean above 38.5 °C; min below 35.5 °C	Fever; hypothermia
Respiration rate	mean below 10 / above 24	Bradypnea / respiratory distress
Systolic pressure	mean below 90 / above 140	Hypotension / hypertension

On closing a window the gateway avoids one message per patient per vital. It assembles all of that cycle's summaries and transmits them together in a single batched call, dividing them into groups only where the queue restricts how many entries a call may hold. Because two patients yield at most ten summaries per cycle, what would be ten cloud calls reduces to one, cutting both request cost and per-message overhead.

C. Queue-Decoupled Ingestion and the Reading Store

The aggregates are not passed straight to the storage function; they are enqueued on a managed message queue instead. That queue reconciles the differing pace of the two sides: it retains messages durably until they are consumed, so a consumer that runs slow or drops out briefly loses nothing and applies no back pressure to the gateway, and a surge of aggregates is paced to a rate the consumer can bear, which is the producer-consumer separation characteristic of message-oriented middleware [8]. That guarantee begins once a message is enqueued and does not extend to the enqueue itself: should the gateway's batched call fail, the window is already sealed and cleared, so the gateway records the failure and continues rather than retrying or holding data back, an accepted compromise given the modest volume two patients generate.

An event-driven ingestion function drains the queue, converts each aggregate into a stored record, and writes it to a managed key-value store under a composite key. The partition key is the vital; collecting every reading for one sign in a single place makes the dominant request, the recent windows for a given vital, resolvable in one lookup. The sort key appends the patient identifier to the window-end timestamp. Ordering by the timestamp keeps each partition chronological, and appending the patient identifier is intentional: two patients closing the same vital in one cycle produce records that coincide on partition key and window-end time, so without the patient identifier the later write would displace the earlier. Appending it keeps the two patients' records separate within a single partition.

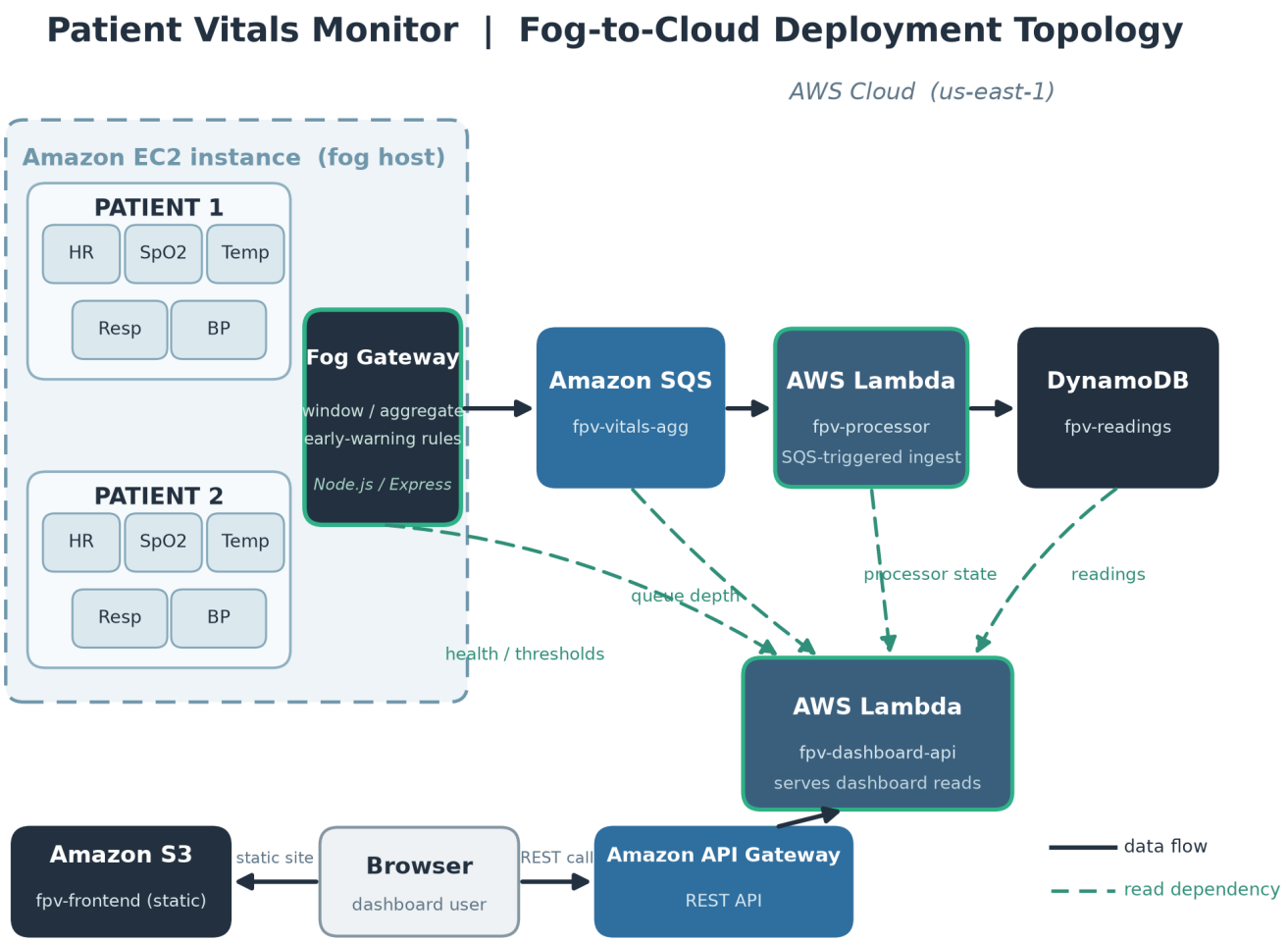

Fig. 1. The deployed fog-to-cloud topology. A single edge host at the bedside runs the fog gateway and ten sensor containers. Ingestion follows the solid path, instruments to queue to ingestion function to store. The dashed paths are the sources the dashboard function reads back, the gateway, the queue, the ingestion function, and the store. The browser retrieves static assets from object storage and reaches the dashboard interface via the managed gateway.

D. Read Tier: Interface, Live Trace, and Ward View

A second cloud function, deployed separately from the ingestion function, serves the clinician's view, which allows reads and writes to scale and to fail in isolation. It exposes a small read-only interface: a patients endpoint returning, per patient, the latest reading and window statistics for each vital; a readings endpoint returning a recent series for one vital and patient, used for the live heart-rate trace; a health endpoint; a back-end statistics endpoint; and a thresholds endpoint that relays the gateway's published rules. Health is returned as four separate flags, whether the gateway responds, whether the queue is reachable, whether the ingestion function is active, and whether the pipeline is current in that the newest stored window is recent, accompanied by the age of that window.

Object storage serves the static frontend, comprising one page, a stylesheet, the page script, and a charting library kept within the project. Each patient appears on the page as a card centred on a live heart-rate trace taken from the readings endpoint and styled after a bedside monitor, with the remaining four vitals as tiles that carry their latest value, their window range, and a flag whenever a threshold is crossed. A ward summary tallies the patients under watch and the vitals presently out of range, and a critical banner names any patient with a firing alert. On a short interval the script requests the patients view, health, and back-end statistics afresh and redraws from each reply. It obtains the interface origin from one inline value in the page, replaced with the real gateway origin when the assets are uploaded; run locally, where a single process serves both interface and page, the value is left empty and requests default to the same origin.

E. Alternatives Considered

Every decision below has a stated rationale.

A queue in place of a direct call. The gateway might call the storage function directly and synchronously, which would remove the queue along with a hop of latency. That was declined: a synchronous call binds the edge to the availability

and speed of the cloud function, so if the function is throttled, cold, or unavailable the gateway stalls or sheds data and the pressure reaches back to the instruments. A queue severs that binding, takes up surges, retries on the consumer's behalf, and lets the gateway regard an aggregate as delivered the moment it is enqueued [8]. On a window cadence the extra latency does not matter; the durability and the separation are what a bedside monitor's link requires.

A key-value store in place of a relational database. The access is narrow and fixed in advance: append aggregates in time order, then retrieve the recent windows for a vital. A key-value store designed for that combination of heavy writes and key-based reads sustains predictable performance as it scales and spares the operational overhead of a relational engine, the balance the original Dynamo work set out and the managed service continues [9], [10]. Under the chosen keys, the dashboard's query becomes a single partition lookup.

Event-driven functions in place of a standing server. Both the ingestion and the interface tier run as event-driven functions, a match for a workload that is bursty and, for the interface, only occasional: billing follows requests rather than idle capacity, and the platform scales the functions with demand. The recognised cost is a cold start once a function has idled [11]; its effect here is small, since steady queue traffic keeps the ingestion function warm and the dashboard's frequent polling does the same for the interface.

F. Security and Privacy Posture

A single inbound port is open on the edge host, and it carries only the gateway's health and threshold endpoints; with no login key pair and all administration over the provider's systems-management channel, the commonest remote-access foothold is removed. The two functions run under the account's shared laboratory role because the account may not create new roles; in production the intent is to divide that into least-privilege roles, allowing the ingestion function only to receive and delete queue messages and write records, and the dashboard function only to read. The telemetry here is simulated and bears no patient identity, only a synthetic patient label, so nothing personal or protected is stored or served; a genuine clinical deployment of the same interface would further require authentication, encryption in transit and at rest, and access control, since vital-sign data is sensitive. Because the page and the interface occupy different origins, every reply, error replies included, carries an explicit cross-origin header so the browser will accept a response returned across origins [12].

III. Implementation

Node.js implements the whole stack, with no build step and no TypeScript. The fog gateway is built with Express, and the queue, store, and function clients are taken from the official JavaScript SDK the provider ships. The dashboard's heart-rate trace uses a charting library committed to the project and served from it rather than fetched from a content delivery network, so no third-party code loads at runtime. In development and in continuous integration the whole stack comes up under Docker Compose over a local emulator of the cloud services, and moving between that emulator and the live account changes only the endpoint and credentials given to otherwise identical code.

TABLE II
Automated Test Coverage by Module

Module	Tests
Sensors	4
Fog gateway	16
Ingestion function	5
Dashboard interface	16
Total	**41**

A. Dashboard Handler and the Runtime Argument Guard

Running the dashboard interface as a cloud function brought two implementation details worth recording. First, its routing takes the form of a small ordered route table, each entry pairing a path with a handler and each request matched against the table in turn, which keeps dispatch declarative and simple to extend with another endpoint. Second, so that it can be tested, the handler takes an optional argument holding ready-made cloud clients, which the unit tests provide as fakes. Because the serverless runtime passes a completion callback among a handler's arguments, the handler treats that argument as injected clients only when it genuinely carries a store client, and builds its own otherwise; the fake path the tests use and the self-building path production needs are told apart by the argument's shape rather than its mere presence, which stops the runtime's callback from being read as injected clients.

B. Automated Test Suite

Testing uses the runner built into the Node.js runtime, with no outside framework. Since cloud-facing code takes its client as a parameter, the unit tests hand it small handwritten fakes instead of a live service or the emulator, which keeps the suite quick, repeatable, and off the network. Across the four modules the suite comes to 41 tests, split as Table II shows, and the dashboard's share includes tests that assert the runtime argument guard described above.

C. Continuous Integration

A continuous-integration workflow fires on every push and pull request as two jobs in sequence. The first installs dependencies and runs the unit suites of all four Node.js modules. The second proceeds only when the first has passed: it raises the emulator-backed Docker Compose stack, runs an end-to-end check that pushes data through the sensors, the gateway, the queue, the ingestion function, and the store, and then dismantles the stack. Placing the integration job behind the unit job keeps ordinary feedback fast, since a logic error is caught by the quick unit suite, yet still exercises the whole assembled system before any change is admitted.

D. Infrastructure-as-Code Deployment

Deployment was to a real provider laboratory account in the North Virginia region. Provisioning is expressed wholly as infrastructure as code and run in an isolated workspace so that the plan accounts only for this project's resources: one apply produced twenty-four resources with no commands entered by hand, the plan listing twenty-four to add and none to change or destroy. The resources are a key-value table, a message queue, the ingestion and dashboard functions reached through a REST interface gateway, an edge compute instance whose firewall opens only the one gateway port and which holds no login key pair, a fixed public address bound to that instance, and two object-storage buckets, one for the dashboard frontend and one for staging the deployment. The application code and the infrastructure-as-code configuration are documented in the repository at [GitHub repository URL].

The system was then evaluated by measuring the running deployment, not by inspecting code alone; all figures below were read from the live endpoint.

IV. Live Evaluation

A. End-to-End Liveness

About a minute after the edge instance completed its container start-up, the health endpoint showed all four indicators true, the gateway reachable, the queue reachable, the ingestion function active, and the pipeline current, with the newest stored window only a few seconds old. When that age is in single digits, a reading taken at the bedside has already crossed the sensor, the gateway, the queue, the ingestion function, and the store and is being served back on the dashboard, the whole passage taking seconds, which is what sets a genuinely connected pipeline apart from a set of components merely deployed alongside each other.

B. Accumulation and a Firing Alert

Polled repeatedly through the check, the stored record count rose from one poll to the next, which confirms the ingestion function was draining the queue and writing on an ongoing basis rather than once. Over the same period the patients endpoint returned live data for both patients, five current vitals each with their window range and mean. During verification one patient's oxygen saturation drifted below the hypoxia threshold, and the pipeline behaved end to end as designed: the edge rule raised a hypoxia alert on the window, the alert propagated through the queue and the store to the read tier, the ward summary counted one vital out of range, and the dashboard's critical banner named the patient and the hypoxia condition while the patient's oxygen-saturation tile was flagged. All static frontend assets were fetched successfully from object storage, the cross-origin header was present on the interface responses, and the inline interface origin had been correctly resolved to the real gateway address during upload.

Fig. 2 is a screenshot of the deployed dashboard rendered in a real browser. It shows both patient cards with their live heart-rate traces and vital tiles, the ward summary reporting one vital out of range, and the critical banner naming the hypoxia alert on the affected patient. The screenshot confirms that the system presents correct, live, clinically annotated content to a clinician, and not merely raw numbers.

C. Cost

Standing cost is low by design. With the exception of the edge instance, every service in the pipeline charges per request and, at these volumes, stays inside the provider's free tier, the queue, the two functions, the key-value store, the interface gateway, and the object storage each billing only on use. The edge instance is the one always-billed resource, and since the serverless dashboard and interface do not require it running, only fresh readings and the gateway's own health check do, it can be paused while idle without dropping the dashboard. Idle, therefore, the system costs little more than one small instance, and that too can be paused.

D. Limitations

Three limitations are worth stating directly. First, the sensor and fog tier occupies a single edge instance and is thus that tier's single point of failure: stop the instance and fresh readings cease, though the already-stored data and the serverless read path stay reachable. A production bedside deployment would replicate the fog tier or place a gateway at each bay. Second, the laboratory account being session based, only single-session behaviour was confirmed; stability across several days and credential rotation over long runs were not put to the test. Third, the two functions share one broadly scoped account role at present; the intended division into least-privilege roles was not possible in an account that cannot create roles.

V. Conclusion

The aim of this work was to carry patient vital-sign monitoring from intermittent observation toward continuous, edge-screened surveillance, on a distributed design that assigns each task its proper place: windowing, statistical reduction, and early-warning checks on a fog tier at the bedside, with durable storage and elastic serving in the cloud. What emerged is a working pipeline over two patients and five vitals each, screening every vital against a two-sided early-warning threshold at the edge and presenting a live bedside-style dashboard served from a real cloud deployment.

A few lessons stand out. The message queue earned its place through resilience rather than latency: separating the bedside from the cloud consumer is what lets the pipeline tolerate a lossy uplink. The sort key showed how much a small modelling choice can carry, since attaching the patient identifier to a timestamp is all that keeps two patients coexisting instead of one quietly overwriting the other. A further lesson concerned the line between test seams and the runtime: a handler that accepts injected clients for testing has to tell them apart from the arguments the serverless runtime supplies on its own, or it will fail in production while clearing every local test. Live verification bore out the design end to end when a real hypoxia excursion travelled from an edge rule to a named alert on the

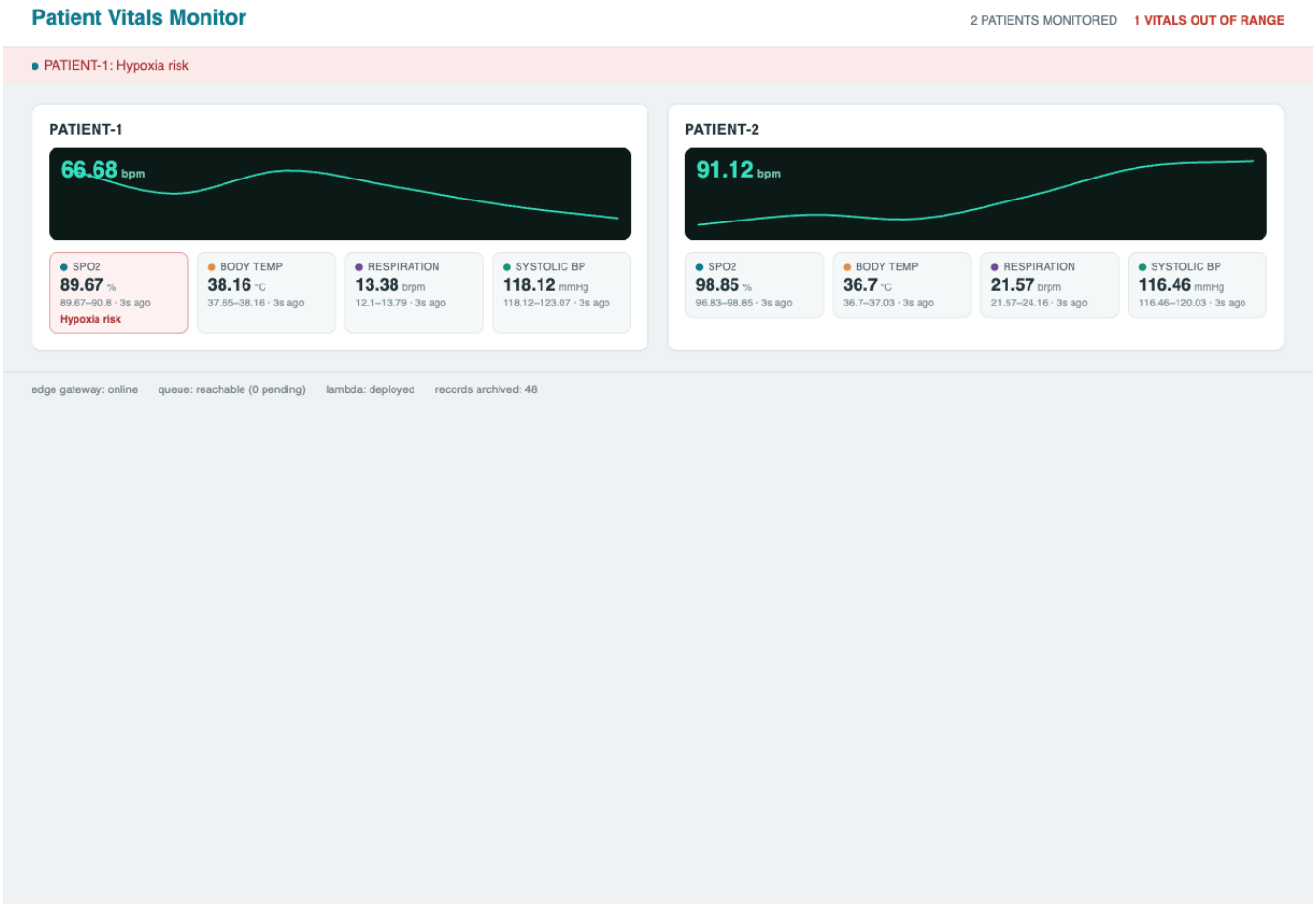

Fig. 2. The deployed operator dashboard rendered live in a browser. Each patient card shows a live heart-rate trace and the other four vitals as tiles with their window range; the header reports two patients monitored and one vital out of range; the banner names the firing hypoxia alert, and the affected oxygen-saturation tile is flagged.

clinician's screen. Sensible next steps are to replicate the fog tier for availability, split the shared role into least-privilege roles, add authentication and encryption for a genuine clinical setting, and move the edge rules from fixed thresholds toward the additive scoring that formal early-warning systems use, so that several mild deviations together raise concern before any single one crosses its line.

References

[1] C. P. Subbe, M. Kruger, P. Rutherford, and L. Gemmel, "Validation of a modified early warning score in medical admissions," *QJM: An International Journal of Medicine*, vol. 94, no. 10, pp. 521–526, 2001.

[2] Royal College of Physicians, "National early warning score (news) 2: Standardising the assessment of acute-illness severity in the NHS," Royal College of Physicians, London, Tech. Rep., 2017.

[3] S. M. R. Islam, D. Kwak, M. H. Kabir, M. Hossain, and K.-S. Kwak, "The internet of things for health care: A comprehensive survey," *IEEE Access*, vol. 3, pp. 678–708, 2015.

[4] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[5] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the internet of things," in *Proc. 1st Edition of the MCC Workshop on Mobile Cloud Computing (MCC '12)*, 2012, pp. 13–16.

[6] R. K. Naha, S. Garg, D. Georgakopoulos, P. P. Jayaraman, L. Gao, Y. Xiang, and R. Ranjan, "Fog computing: Survey of trends, architectures, requirements, and research directions," *IEEE Access*, vol. 6, pp. 47 980–48 009, 2018.

[7] P. Mell and T. Grance, "The NIST definition of cloud computing," National Institute of Standards and Technology, Tech. Rep. Special Publication 800-145, 2011.

[8] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003.

[9] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels, "Dynamo: Amazon's highly available key-value store," in *Proc. 21st ACM SIGOPS Symp. Operating Systems Principles (SOSP '07)*, 2007, pp. 205–220.

[10] M. Elhemali, N. Gallagher, N. Gordon, J. Idziorek, R. Krog, C. Lazier, E. Mo, A. Mritunjai, S. Perianayagam, T. Rath, S. Sivasubramanian, J. C. Sorenson, S. Sosothikul, D. Terry, and A. Vig, "Amazon DynamoDB: A scalable, predictably performant, and fully managed NoSQL database service," in *Proc. USENIX Annual Technical Conf. (USENIX ATC '22)*, 2022, pp. 1037–1048.

[11] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, "Peeking behind the curtains of serverless platforms," in *Proc. USENIX Annual Technical Conf. (USENIX ATC '18)*, 2018, pp. 133–146.

[12] A. Barth, "The web origin concept," Internet Engineering Task Force, Tech. Rep. RFC 6454, 2011.